P R I S M C O R E C O D E A U D I T

Аудит ядра PRISM

Громкие и тихие баги, расхождения с математикой, инженерные решения

Объект: github.com/survivorm/prism · prism/ · 29 модулей · 7 373 строки

Сверка: paper.md v5.0, ref-math.md (индекс символов v6.0)

Метод: полное чтение, аналитическая и численная верификация

Дата: 3 ноября 2026 г.

Оглавление

1. Резюме аудита....1
2. Методология и объём....2
3. Громкие баги (подтверждены численно)....2
3.1. B1 — офф-на-один слой в кэше скрытых состояний....3
3.2. B2 — пер-позиционные Якобианы с утечкой будущего....3
3.3. B3 — KL-поводок: в 128 раз слабее спецификации и не тот протокол....4
3.4. B4 — R_can на неправильном срезе....5
3.5. B5 — транспонированный оптимум в procrustes_gap....5
4. Тихие баги и паперка́ты....6
5. Расхождения кода с математикой и статьёй....6
5.1. M1 — оптимизируемый лосс не равен метрике....6
5.2. M2 — арифметическая ошибка в paper.md....6
5.3. M3 — ε в косинусе....7
5.4. M4 — кросс-сэмпловый пул в E-14....7
5.5. M5 — банковское округление в маппинге слоёв....7
5.6. M6 — MRoPE: [4,B,S] в статье против 3-компонентного rotary в коде....7
6. Инженерные решения и запахи....8
6.1. E1 — двойной полный форвард на каждый Якобиан....8
6.2. E2 — лишний полный форвард на KL каждый шаг....8
6.3. E3 — градиенты по всем параметрам....8
6.4. E4 — дублирование вычислительных путей....9
6.5. E5 — monkey-patch GDN-ядер....9
6.6. E6 — монолитный импорт....9
6.7. E7 — T-серия без потребителей и тестов....9
7. Что сделано хорошо....10

8. Рекомендации.....10

Примечание: оглавление построено на кодах полей. После правок документа обновите его: правый клик по оглавлению → «Обновить поле».

1. Резюме аудита

Проведён аудит ядра проекта PRISM — пакета `prism/` (29 модулей, 7 373 строки) из репозитория github.com/survivorm/prism, ветка main. Код сверялся с математической спецификацией (`ref-math.md`, v6.0) и текстом статьи (`paper.md`, v5.0). Все ключевые формулы проверены аналитически; четыре подозрения подтверждены численным экспериментом на миниатюрной модели Qwen3.5 и на синтетике; запущены собственные тесты репозитория (114 тестов в проверенных подмножествах — все проходят) и реестр инвариантов M0 (17/17).

Общий вердикт: математическое ядро (`p_algebra`, `operator_checks`, `holonomy`, `spectral`, `nulls_and_stats`) реализовано аккуратно и почти всюду соответствует бумаге; дисциплина fail-loud и реестр фальсифицируемых инвариантов — редкость для исследовательского кода. Однако найдено **5 громких багов** (два из них — систематическое искажение вычисляемых объектов в инфраструктуре T-серии), **17 тихих** дефектов и паперка́тов, **6 расхождений с математикой/бумагой** и ряд инженерных решений с кратным перерасходом вычислений. Ни один из громких багов не покрывается существующими тестами.

Сводка находок:

ID	Серьёзность	Место	Суть
B1	Высокая	`interventions.py:626`	Кэш h хранит выход слоя L−1 вместо выхода слоя L: реконструкция J из кэша даёт Якобиан «не того» слоя (расхождение 10,7% на модели-проверке)
B2	Высокая	`_exp_common.py`	Пер-позиционные Якобианы при t < S−1 нарушают контракт compute_jacobian_for_tokens: утечка будущих токенов в KV-кэше, неверная RoPE-позиция (расхождение до 28,7%)
B3	Высокая	`interventions.py:242`	KL-поводок в make_readout_pull: batchmean на 1-D тензоре делит на K (поводок в 128 раз слабее спецификации), KL по K-подмножеству вместо full-vocab, протокол расходится с ядром exp16
B4	Средняя	`profiles.py:41`	R_can считается на срезе (I−P)J вместо канонического J(p) = PJ: R_can ≈ R_raw, диагностика «доли калибровки» мертва
B5	Средняя	`exp38_hi_common.py:154`	procrustes_gap использует транспонированный оптимум Прокруста: ненулевой «зазор» для одного подпространства в разных базисах
M1 – M6	Спорные	код vs paper.md	Лосс обучения ≠ метрика (взвешенность), арифметическая ошибка в примере PR в бумаге, токен-сет «union» vs «top-K по частоте» и др.

ID	Серьёзность	Место	Суть
Q1–Q17	Низкая	по всему ядру	Краевые случаи, противоречивые докстринги, дубли имён, конвенции углов θ и др.
E1–E7	Эффективность	jacobian/ interventions	Двойные полные форварды на каждый Якобиан, лишний форвард на KL каждый шаг оптимизации и пр.

Таблица 1. Сводка находок аудита. B — громкие баги, M — расхождения с математикой/бумагой, Q — тихие дефекты, E — инженерные решения.

Зона поражения. Баги B1–B3 живут в «обвязке» T-серии экспериментов (`apply_edit_and_cache` + `_exp_common.py`, `make_readout_pull`). Сами скрипты T0/eps/T2/T1/delta в репозиторий не закоммичены, поэтому уже опубликованные числа (E-серия, снятая с плотного кэша exp27b и с Kaggle-ядер) этими багами **не затронуты** — но любой следующий запуск T-серии через это API получит систематически искажённые объекты. B4 и B5 искажают диагностические числа E-40/E-38h уже в закоммиченном коде.

2. Методология и объём

- Полное чтение всех 29 модулей `prism/` (7 373 строки): jacobian.py (973), interventions.py (704), holonomy.py (594), cache.py (535), pipeline.py (543), invariants.py (344), spectral.py (399) и остальные.
- Сверка каждой формулы с `ref-math.md` (индекс символов, глоссарий, доказательства [M-1]–[M-5]) и `paper.md` §2 (аппарат): оператор P(p), gap/bridge, ESG, голономия, тройка (c, s, frac_K), нуль-модели, секулярное уравнение, MP-край, кинетическая декомпозиция.
- Численная верификация подозрений: мини-модель Qwen3.5 (8 слоёв, D=64, GDN+full-attn) для B1/B2; синтетика для B4/B5, тождества Прокруста и конвенций κ; прямая проверка семантики `F.kl_div` для B3.
- Запуск собственных тестов: `test_p_algebra`, `test_operator_invariants`, `test_spectral*`, `test_transport_kinetic`, `test_esg`, `test_bridge`, `test_holonomy*` (87 passed) и `test_jacobian` + `test_interventions` (27 passed); реестр `prism.invariants` — 17/17.
- Вне объёма: каталог `experiments/` (просмотрен выборочно — только там, где он потребляет ядро), Kaggle-ядра (просмотрен exp16_tpu), данные и результаты.

3. Громкие баги (подтверждены численно)

3.1. B1 — офф-на-один слой в кэше скрытых состояний

Место: `prism/interventions.py:626`, функция `_cache_edit`: `h_t = h_states[L][0]`.

Суть. В transformers `hidden_states[0]` — эмбеддинги, `hidden_states[l+1]` — выход слоя l. Кэш `apply_edit_and_cache` для метки «слой L» сохраняет `hidden_states[L]` — выход слоя L−1, то есть вход слоя L. При этом контракт потребителя, заявленный в докстринге самой функции («reconstruct via compute_jacobian_for_tokens(h_override=h_t[l][t])»), требует выход слоя L: `compute_jacobian_for_tokens(layer_idx=L)` извлекает `hidden_states[L+1]` (`jacobian.py:155`) и реплеит слои L+1…n. Смешение даёт объект «точка из слоя L−1 + хвост со слоя L+1»: слой L пропущен в линеаризации, точка — чужая.

Внутреннее противоречие. В одном файле `experiments/_exp_common.py` используются обе конвенции: `top_r_basis` и `build_edits` берут `hidden_states[L+1]` (строки 86 и 102 — правильно), а `doc_js` получает h из дискового кэша (строка 144 — слой L−1). То есть базис ESG и сами Якобианы в одном эксперименте считаются на соседних слоях.

Доказательство. На мини-модели (L=2, S=8): `h_cached[L][t] − hidden_states[L][t] = 0` (точно), `||h_cached − hidden_states[L+1]|| = 0.027`; Якобиан, восстановленный из кэша, отличается от прямого на **10,7%** (относительная норма Фробениуса). На полной 24-слойной модели с более глубоким хвостом эффект будет не меньше.

Почему тесты не ловят. `test_apply_edit_and_cache_roundtrip` проверяет только формы, dtype и перенормировку p — не тождество слоя.

Исправление. Одна строка: `h_t = h_states[L + 1][0]`. Плюс регрессионный тест: кэш [L] должен совпадать с `hidden_states[L+1]` и давать J, равный прямому вычислению.

3.2. B2 — пер-позиционные Якобианы с утечкой будущего

Место: `experiments/_exp_common.py:85–88, 143–145` (`top_r_basis`, `doc_js`); корень — контракт `compute_jacobian_for_tokens` (`jacobian.py:131–142, 354`).

Суть. `compute_jacobian_for_tokens` жёстко привязан к последней позиции: префикс-кэш строится из `input_ids[:, :-1]` (позиции 0…S−2), RoPE-позиция — S−1. Хелперы же гоняют его по последним n_pos=8 позициям, подставляя `h_override`

`= h[t]` при t < S−1. Для t < S−1 такой вызов: (а) подмешивает в KV-кэш токены t…S−2 — будущее относительно t; (б) ставит RoPE-позицию S−1 вместо t; (в) использует калибровку p[t] (softmax позиции t), не согласованную с фактической точкой линеаризации. Докстринг `compute_jacobian_for_tokens` прямо относит согласованность h_override с input_ids на ответственность вызывающего — но вызывающие в том же репозитории её нарушают.

Доказательство. Мини-модель, L=2: корректный J_t (префикс 0…t−1, позиция t) против «как в doc_js»: расхождение **18,7%** при t=S−2 и **28,7%** при t=S−3. Все пер-позиционные ESG/метрики, построенные через этот путь, смешивают позиции.

Исправление. Передавать `input_ids[:, :t+1]` и `attention_mask[:, :t+1]` вместе с h[t] (тогда префикс и позиция корректны автоматически), либо добавить в `compute_jacobian_for_tokens` явный параметр position. Это также устранит несогласованность p[t].

3.3. B3 — KL-поводок: в 128 раз слабее спецификации и не тот протокол

Место: `prism/interventions.py:242` (`make_readout_pull`), строка `kl = F.kl_div(p_edit.log(), p_base, reduction="batchmean")`.

Суть (три дефекта в одной строке и вокруг). Во-первых, `p_edit` — одномерный тензор [K]; для 1-D входа `batchmean` делит сумму на `input.size(0) = K`, поэтому kl = KL/K. При K=128 и весе 0.1 эффективный коэффициент поводка — **0.1/128 ≈ 7.8×10⁻⁴** вместо 0.1 (проверено численно: batchmean == sum/K). Во-вторых, KL считается по ренормированному K-подмножеству словаря, тогда как ref-math (статья «leash», E-16a) требует full-vocab KL: поводок слеп к перетеканию массы между token set и остальным словарём. В-третьих, сам протокол расходится с ядром, породившим опубликованные числа E-16a: `kaggle_kernel/exp16_tpu/exp16_readout_pull.py:1501` считает честный full-vocab KL (batchmean на [1, V], log_target=True), но не маскирует градиенты — Adam через KL двигает **все** строки lm_head, тогда как `make_readout_pull` маскирует до token_set. То есть «E-16a протокол» в ядре пакета и в Kaggle-ядре — **два разных эксперимента** (в ядре доза α* = ‖ΔW‖/‖W‖ мерится по полному W).

Последствия. Докстринг «E-16a protocol» в `make_readout_pull` вводит в заблуждение: воспроизведение E-16a через core даст другую траекторию (слабый поводок + только K строк). Выводы о «KL-поводке держит распределение» из core-прогона были бы необоснованными.

Исправление. `reduction="sum"` (или добавить ось батча), logits полного словаря, и явное решение: либо маскировать градиенты как в T3, либо не маскировать как в ядре — и зафиксировать это в докстринге и в статье.

3.4. B4 — R_can на неправильном срезе

Место: `prism/profiles.py:41–42`: `Kmat = J − apply_P(J, p); return _lam_share(Kmat @ Kmat.T)`.

Суть. По бумаге (§2.5, §2.11.1) R_can = λmax/tr(E_K), где `E_K = J^(p) (J^(p))^T` — грамиан канонического (gauge-free) среза `J^(p) = P(p)J` (fixP). Код же строит срез `(I−P)J` — «убитую» компоненту K_t из E-37. Поскольку frac_K ≈ 0.998 ((I−P)J ≈ J по энергии), кодовый R_can практически совпадает с R_raw: на синтетике с выраженной gauge-модой код дал R_can = 0.9988 против R_raw = 0.9988 (разница 3.6×10^{-5}), тогда как каноническое определение даёт 0.395. Диагностика «R_raw − R_can = доля калибровки в топ-моде», заявленная в статье, таким кодом мертва: разница всегда ≈ 0.

Исправление. `Kmat = apply_P(J, p)` (и оставить `frac_K` на срезе (I−P)J, как в E-37, — это разные объекты, у каждого своё место).

3.5. B5 — транспонированный оптимум в procrustes_gap

Место: `prism/exp38_hi_common.py:152–156`.

Суть. Докстринг: «||Q1 − Q2 O||_F / r over orthogonal O» — то есть минимум по O. Минимум даёт O* = polar($Q2^{T}Q1$); код же строит SVD матрицы `Q1ᵀQ2` и берёт `O = U @ Vt = polar(Q1ᵀQ2) = O*ᵀ` — транспонированный оптимум. Следствие: для двух фреймов **одного и того же подпространства** в разных базисах (Q2 = Q1·R, $R^2 \neq I$) функция возвращает ненулевой «зазор». Численно: код 0.499 против истинного минимума 0.405 на случайных фреймах. Числа E-38h «frame alignment», посчитанные этой функцией, завышены/неинтерпретируемы как прокрустовы расстояния.

Исправление. Поменять порядок операндов: `U, _, Vt = svd(Q2.T @ Q1); O = U @ Vt`.

4. Тихие баги и паперка́ты

Ниже — дефекты, которые не искажают опубликованные результаты напрямую, но либо ломаются на краевых случаях, либо создают ловушки согласованности. Отсортированы по предполагаемому риску.

ID	Место	Суть
Q1	`svd_bundle.py:26`	effective_rank(нули) возвращает полный размер (threshold·σ_1 = 0, все σ ≥ 0 проходят) вместо 0. Краевой случай нулевого J.
Q2	`spectral.py:351`	Докстринг gauge_share: «Distinct from g_uni» — противоречит p_algebra.g_uni («identical by construction, O2≡O3») и бумаге E-39: на одном J это один функционал. Устаревший текст до E-39.
Q3	`tag.py:50`	TAG_DOMAINS допускает aggregation="energy_weighted", а operator_checks._agg его отвергает с подсказкой «equals ratio_of_means — use it». Тег легален, агрегатор — нет.
Q4	`bridge.py:81–100`	_gramian_key: хешируются 9 средних строк, а не 10 (заявлено в докстринге и в paper §2.2); строки 0 и V−1 хешируются дважды; device входит в ключ (дублирование кэша CPU/GPU). Риск коллизий репрезентативного хеша в бумаге честно оговорён.
Q5	по всему ядру	Одноимённые функции с разной семантикой: isotropic_reference (p_algebra — от p; operator_checks — от матрицы O), gauge_share (p_algebra — от грамиана; spectral — от спектра+векторов), kappa (transport = ‖log H‖_F; holonomy = ‖H−I‖_F — разводка документирована в ref-math, но ловушка имени остаётся).
Q6	`holonomy.py:214`	Конвенции углов не сведены: holonomy_spectrum возвращает r/2 углов (по одному на пару ±θ), transport.holonomy_angles — r углов (оба знака). Обе корректны, но потребитель, суммирующий cos для tr(H), обязан знать, какую из них держит.
Q7	`holonomy.py:370`	eigenframe_basis не фиксирует знаки базиса по det = +1, как требует конвенция [D-holonomy-frame] из ref-math («знаки — по детерминанту +1»). Компонентное разложение H остаётся неоднозначным.
Q8	`exp14_snr_pr.py:191`	all_results["params"] = {model: model, …, K: K, …} — «голые» переменные как ключи dict: JSON параметров искажён, при K=64 и max_length=64 ключи коллидируют.
Q9	`calibration.py:112–130`	Путь с offset_mapping токенизует с add_special_tokens=False, обычный путь — с default True: одна и та же функция даёт разные токенизации.
Q10	`calibration.py:136–158`	build_global_token_set: бумага говорит «union top-K», код берёт K самых частых токенов пер-сэмпловых top-K (совпадает с union только при K ≥ |union|); ref-math при этом требует «размер K» — терминология спецификации внутренне противоречива.
Q11	`kinetic.py:191`	channel_energy_matrix зовёт transport_log без защит kinetic_decomposition (нет try/except, нет проверки linearization_valid) — в одном модуле разная робастность.
Q12	`jacobian.py:253`	Контрольная проверка клона кэша — assert: исчезает при python -O, превращаясь из «громкой» в молчаливую.
Q13	`bridge.py:9`	Кэши и результаты пишутся в относительные пути (results/.gramians, results) — зависят от CWD запуска.

ID	Место	Суть
Q14	`bridge.py:8`	_gramian_cache — неограниченный in-memory dict без инвалидции (для 1–2 моделей приемлемо, для длинных процессов — утечка по дизайну).
Q15	`interventions.py:321, 384`	make_random_ortho/matched создают G на CPU независимо от устройства delta_W_ref (спасает .to(weight) в apply_edit — но устройство теряется в пост-проверках).
Q16	`nulls_and_stats.py:92`	reduction_control: при константной статистике (ss = 0) $r^2 = 1.0$ — спорная конвенция (всё «объяснено» чем угодно).
Q17	`exp38_hi_common.py:146`	overlap(Q1, Q2, r): параметр r не используется (мёртвый аргумент).

Таблица 2. Тихие дефекты. Q4–Q6, Q10 — документационные/конвенционные ловушки; Q1, Q8–Q9, Q16–Q17 — кодовые паперка́ты.

5. Расхождения кода с математикой и статьёй

5.1. M1 — оптимизируемый лосс не равен метрике

`bridge_rows_backward` (`jacobian.py:863`) и `make_readout_pull` оптимизируют невзвешенное среднее (1/K)Σ(1−cos) по строкам. Метрика же в статье — `ϖ_gap(l) = Σ w_i(1−cos)` с весами w = p_L (нормировка на ∑). Для E-16a бумага сама пишет невзвешенную форму, но P2-лосс (§7, Exp 17–19) определён взвешенным. В результате Exp 17 оптимизирует одну целевую функцию, а отчётные числа моста считаются по другой. Это не ошибка вычислений, а недокументированная дивергенция «objective vs metric» — в тексте статьи она нигде не оговорена.

5.2. M2 — арифметическая ошибка в paper.md

В §2.3 статьи пример разведения рангов: для λ = (4,1,1,1,1) утверждается «PR = 64/18 ≈ 3.6». На самом деле $\Sigma\lambda^2 = 20$, PR = 64/20 = **3.2**. Код (докстринг `spectral.stable_rank`) прямо документирует эту опечатку T3 и считает правильно — здесь код прав, а статья нет; в статье стоит поправить.

5.3. M3 — ε в косинусе

Формула §2.4: знаменатель `‖J‖·‖J_target‖ + ε` (добавление ε). Код: `F.cosine_similarity` — clamp-мин 10^{-8}. Для живых строк численно эквивалентно, формально — разные объекты; в sentinel-режиме (threshold = 0) поведение на почти нулевых строках чуть различается. Косметика, но при численной верификации «формула-в-коде» она всплывает.

5.4. M4 — кросс-сэмпловый пул в E-14

`exp14_snr_pr.py` пулит Якобианы N разных документов в один ESG через `compute_esg`, тогда как докстринг `esg.py` называет кросс-сэмпловый пул невалидным (E-0.1: C_b ≈ 0.003–0.05, почти ортогонально). Бумага оговаривает это как «допустимое приближение в гладких слоях» — но сам API `compute_esg` ничего не проверяет и молча позволяет невалидное использование (флаг trajectory_id или явный параметр pool= закрыл бы ловушку).

5.5. M5 — банковское округление в маппинге слоёв

`uniform_layer_mapping` использует `round` (Python: round-half-to-even). Детерминированно, но на ровных значениях 0.5 даёт неинтуитивные соответствия (32→24: слои с i·23/31 = k+0.5 уйдут «вниз» для чётных, «вверх» для нечётных). Для воспроизводимости достаточно задокументировать; для честности — взять floor(x+0.5).

5.6. M6 — MRoPE: [4,B,S] в статье против 3-компонентного rotary в коде

Статья (§3.1): «position_ids формы [4, B, S] (text, temporal, height, width); для текстового входа все 4 компоненты одинаковы». Код (`jacobian.py:354–357`) строит 4-компонентный тензор и передаёт rotary-эмбеддингу только pos[1:] (3 компоненты), а pos[0] — в маски. Численно всё корректно (все компоненты равны S−1), но описание в статье неточно относительно фактического API модели.

6. Инженерные решения и запахи

6.1. E1 — двойной полный форвард на каждый Якобиан

`compute_jacobian_for_tokens` при prefix_cache=None выполняет `_run_prefix_pass` (S−1 токенов, все слои) и `_extract_hidden_state` (S токенов, все слои) — примерно вдвое лишней работы на каждый вызов. Один форвард с use_cache=True и output_hidden_states=True + обрезка кэша до S−1 дал бы и h, и префикс. Сверху `run_layer_diagnostics` добавляет третий полный форвард ради логитов последней позиции. При протоколе «6 слоёв × N сэмплов» это кратный перерасход.

6.2. E2 — лишний полный форвард на KL каждый шаг

В `make_readout_pull` KL-слагаемое каждый шаг запускает полный форвард модели, хотя p_edit на последней позиции — это просто softmax(`h_norm @ W[ts]ᵀ`), а h_norm уже посчитан внутри `compute_hidden_jacobian_rows`. Двойной перерасход на шаг × 100 шагов, плюс второй автограф-граф.

6.3. E3 — градиенты по всем параметрам

В `make_readout_pull` (и в Kaggle-ядре exp16) `loss.backward()` строит градиенты по всем обучаемым параметрам модели, хотя оптимизатор содержит только lm_head: лишняя память и время, а рассуждающий «заморозил — значит, дёшево» получит сюрприз. В exp17 заморозка делается явно (requires_grad = False) — там корректно.

6.4. E4 — дублирование вычислительных путей

Реплей хвоста декодера реализован трижды (`_run_decoder_pass`, `_run_decoder_to_norm`, через `_forward_h_norm`) с чуть разным доступом к конфигу; `isotropic_reference` и `gauge_share` живут по два раза (torch/numpy); голономия — два независимых стека (torch `holonomy.py` и numpy `transport.py` с разной обработкой знаков); `gauge_free_J` — две версии (`_exp_common` и `_e38_cache`, вторая корректная); `uniform_layer_mapping` — две копии (одна задокументирована как сознательная для инлайнинга в Kaggle-ноутбук). Для кодовой базы, чей главный актив — воспроизводимость, каждый дубль — будущий расползшийся фикс.

6.5. E5 — monkey-patch GDN-ядер

`_clone_gdn_states_on_entry` подменяет модульные атрибуты transformers (`torch_recurrent_gated_delta_rule`, `causal_conv1d_update`) с глобальным счётчиком глубины: не потокобезопасно, молча no-op'ится при смене способа связывания ядер (сам код это признаёт и полагает, что тогда ошибка вернётся «громко»). Приемлемый компромисс для CPU/однопотока, но точечный апгрейд transformers превращается в минное поле.

6.6. E6 — монолитный импорт

`import prism` тянет torch + transformers + datasets + matplotlib одномоментно (диагностика импортирует matplotlib на уровне модуля). Для datasets это задокументировано как сознательное решение; matplotlib туда же не входит в аргументацию.

6.7. E7 — T-серия без потребителей и тестов

Pipeline объявляет `_VALID_EXPERIMENTS = (T0, eps, T2, T1, delta)`, `_exp_common.py` содержит полный набор хелперов — но ни одного экспериментального скрипта T-серии в репозитории нет и тестов на хелперы нет. Именно в этом непротестированном слое и сидят B1–B3. Правило «не коммитить обвязку без хотя бы smoke-теста потребителя» здесь бы всё предотвратило.

7. Что сделано хорошо

— **Реестр инвариантов M0 (`invariants.py`)**: 17 фальсифицируемых проверок с негативными контролями (спектральное тождество κ обязано ломаться на θ = 0/π; смешанные агрегации обязаны рвать тройное тождество). Все 17 проходят. Это редчайший уровень дисциплины для исследовательского кода.

— **Урок E6 закодирован в API**: тег канала (`tag.py`) делает slice/aggregation/p_mode identity-полями, агрегатор запрещает смешение медианы и отношения сумм — та самая ошибка, из-за которой «фантомное $s^2 < 0$» жило в статье месяцами, теперь структурно невозможна.

— **Ядро математики верное**: применение P(p) (обе реализации, torch и numpy, формульно совпадают), Procrustes-выравнивание и тождество residual = $2r - 2\Sigma\sigma$ (проверено), κ-инвариантность голономии к смене базиса, g_uni ≡ gauge_share (O2≡O3, 7.7×10^{-16}), секулярное уравнение P(p) с честным бисекционным солвером, MP-край, Westfall–Young и кластерная перестановочная статистика, KL_lin с фактором Фишера, аналитический Якобиан последнего слоя (формула [L-rms] выведена правильно, включая порядок diag(g)), кинетическая 6-канальная бухгалтерия с знаковым кросс-членом.

— **Fail-loud последовательно**: валидация входов почти в каждой функции, атомарные записи (tmp + os.replace), проверки консистентности кэша (meta/dtype/torch-версия), отказ «молча вернуть None» для битых файлов.

— **Дисциплина восстановления состояния**: apply_edit возвращает веса в finally, deterministic_mode аккуратно восстанавливает RNG-состояния всех трёх библиотек, run_spec пишет meta.json до и после с диагностикой крашей.

— **Честная числовая генеалогия**: snr_e14 воспроизводит закоммиченные результаты E-14 бит-в-бит; в докстрингах фиксируются опечатки T3 (PR 64/18 → 64/20, инвертированные метки GDN-слоёв в exp38a).

8. Рекомендации

Приоритет P0 (до любых новых прогонов T-серии):

- B1: `h_t = h_states[L + 1][0]` + регрессионный тест на слой-идентичность кэша и совпадение J с прямым вычислением.
- B2: в `doc_js`/`top_r_basis` передавать `input_ids[:, :t+1]` (или добавить параметр position в compute_jacobian_for_tokens); p[t] после этого согласован по построению.
- B3: `reduction="sum"` + full-vocab logits; синхронизировать протокол с Kaggle-ядром (маскирование строк) и переименовать/уточнить «E-16a protocol» в докстринге.
- B4: `Kmat = apply_P(J, p)` в `profiles.R_can`; добавить инвариант «R_can((I−P)-срез) ≈ R_raw» как негативный контроль.
- B5: поменять порядок операндов SVD в `procrustes_gap` + юнит-тест «один subspace в разных базисах → gap = 0».

Приоритет P1 (корректность и защищённость):

- M1: либо перейти на взвешенный лосс в P2, либо явно зафиксировать в статье расхождение objective/metric.
- Тест на контракты h_override (позиция + слой) — сейчас контракт живёт только в докстринге.
- Q1, Q3, Q8: краевые случаи и мелкие фиксы (effective_rank нулей; убрать energy_weighted из TAG_DOMAINS; ключи params в exp14).
- Q6: свести конвенции углов к одной (r/2 с парами) и добавить адаптер с предупреждением.

Приоритет P2 (эффективность и сопровождаемость):

- E1–E3: один форвард на вызов (use_cache + crop), reuse h_norm для KL, заморозка параметров до backward.
- E4: консолидировать дубли (один реплей хвоста, один gauge_free_J, один стек голономии).
- Докстринги Q2/Q4/Q10 и арифметика M2 в paper.md.

Все численные проверки аудита воспроизводимы: скрипты проверки сохранены рядом с отчётом (audit_checks.py, audit_offbyone.py + per-position проверка). Тесты запускались на CPU, torch 2.15, transformers из ветки, где доступен Qwen3_5 (мини-модель собирается конфигом, без загрузки весей).